

Алгоритмизация и программирование

Лекция 2 (C++)

Базовые сведения

Автор языка: Бьярне Страуструп (Bjarne Stroustrup) – датский учёный в области информатики, родился 30 декабря 1950 года в Орхусе (Дания).

Начал разработку языка в 1979 году в Bell Labs (AT&T).

Цель – соединить эффективность и низкоуровневый контроль С с абстракциями высокого уровня.

Изначально язык назывался «C with Classes» – расширение С классами и другими возможностями ООП.

В 1983 году язык получил имя C++.



Характеристика языка

Мультипарадигменный язык: процедурное, объектно-ориентированное, обобщённое (generic), функциональное программирование.

Типизация: статическая; в целом сильная (строгая), но есть неявные преобразования.

Язык компилируемый - компилируется в машинный код.

Ручное управление памятью + умные указатели.

Принцип: за то, что не используешь, не платишь (zero-overhead principle).

Кроссплатформенность: Windows, Linux, macOS, embedded.

Активно развивается комитетом ISO WG21.

История и стандарты

Год	Стандарт	Ключевые возможности
1998	C++98	Первый ISO-стандарт: STL, шаблоны, исключения
2003	C++03	Исправления, уточнения (TC1)
2011	C++11	auto, лямбды, rvalue-ссылки, move, nullptr, thread, regex
2014	C++14	Обобщённые лямбды, constexpr-улучшения
2017	C++17	structured bindings, if constexpr, optional, variant, filesystem
2020	C++20	концепты, ranges, корутины, модули, three-way comparison
2023	C++23	std::expected, std::print, deducing this, import std
2026	C++26	в разработке: рефлексия, контракты, executors и др.

Релиз нового стандарта – каждые 3 года. «Крупные» выпуски (с существенными новыми возможностями) чередуются с «полировочными»: C++20 и C++26 — крупные, C++23 — доработка.

Публичные драфты стандарта C++

Официальные стандарты ISO платные. Но комитет WG21 публикует свободные рабочие драфты – они практически идентичны финальным стандартам (отличаются только редакционными правками).

Основной репозиторий: <https://github.com/cplusplus/draft>

Онлайн-версия текущего драфта (живой документ): <https://eel.is/c++draft/>

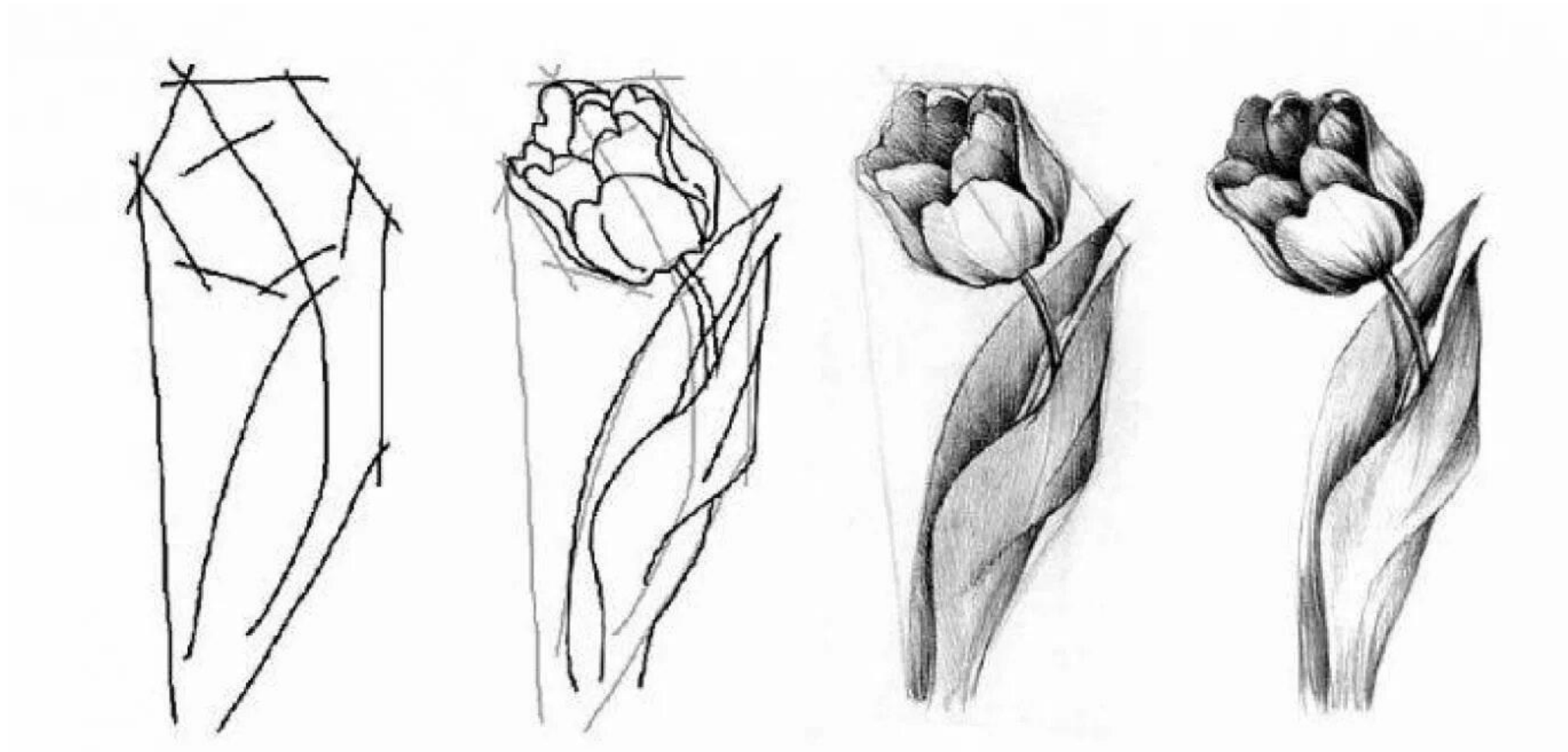
Тут тоже список драфтов: <https://en.cppreference.com/cpp/links>

Нумерованные драфты соответствуют стандартам:

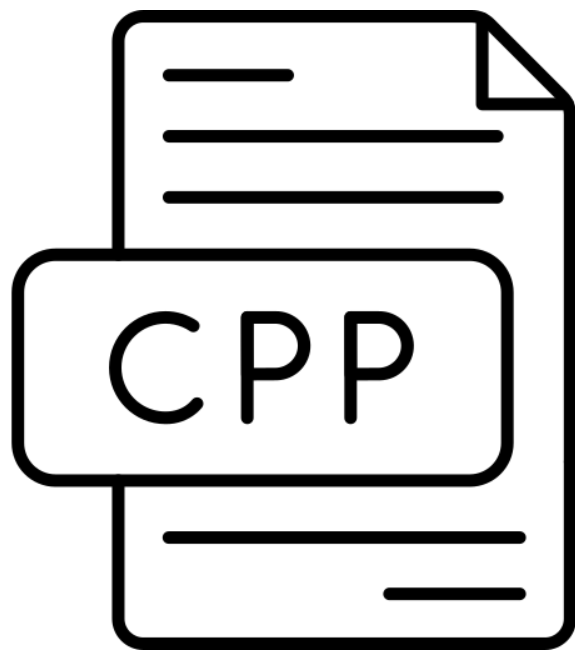
Драфт	Стандарт	Ссылка
N3337	C++11	https://wg21.link/n3337
N4140	C++14	https://wg21.link/n4140
N4659	C++17	https://wg21.link/n4659
N4861	C++20	https://wg21.link/n4861
N4950	C++23	https://wg21.link/n4950

Первая модель программы на C++

Модели



Только один файл



Весь код программы
полностью в одном файле

Структура программы

Программа состоит строго из 2х секций:

```
1  #include <iostream>
2
3
4
5  int main() {
6      // your code goes here
7
8
9
10 }
```

В этой секции только подключаем стандартные библиотеки, если нужно.

Весь код пишем только в этой секции. Эта секция обязательна.

Минимальная программа

```
int main(){} // меньше нельзя
```

```
int main() {  
    return 0; // по стандарту можно не писать  
}
```

Подключение нескольких библиотек

```
#include <iostream>
#include <cmath>
// #include <имя библиотеки>

int main(){
    // код
}
```

Комментарии

Комментарии можно писать в любом месте файла. Они не влияют на результат работы программы.

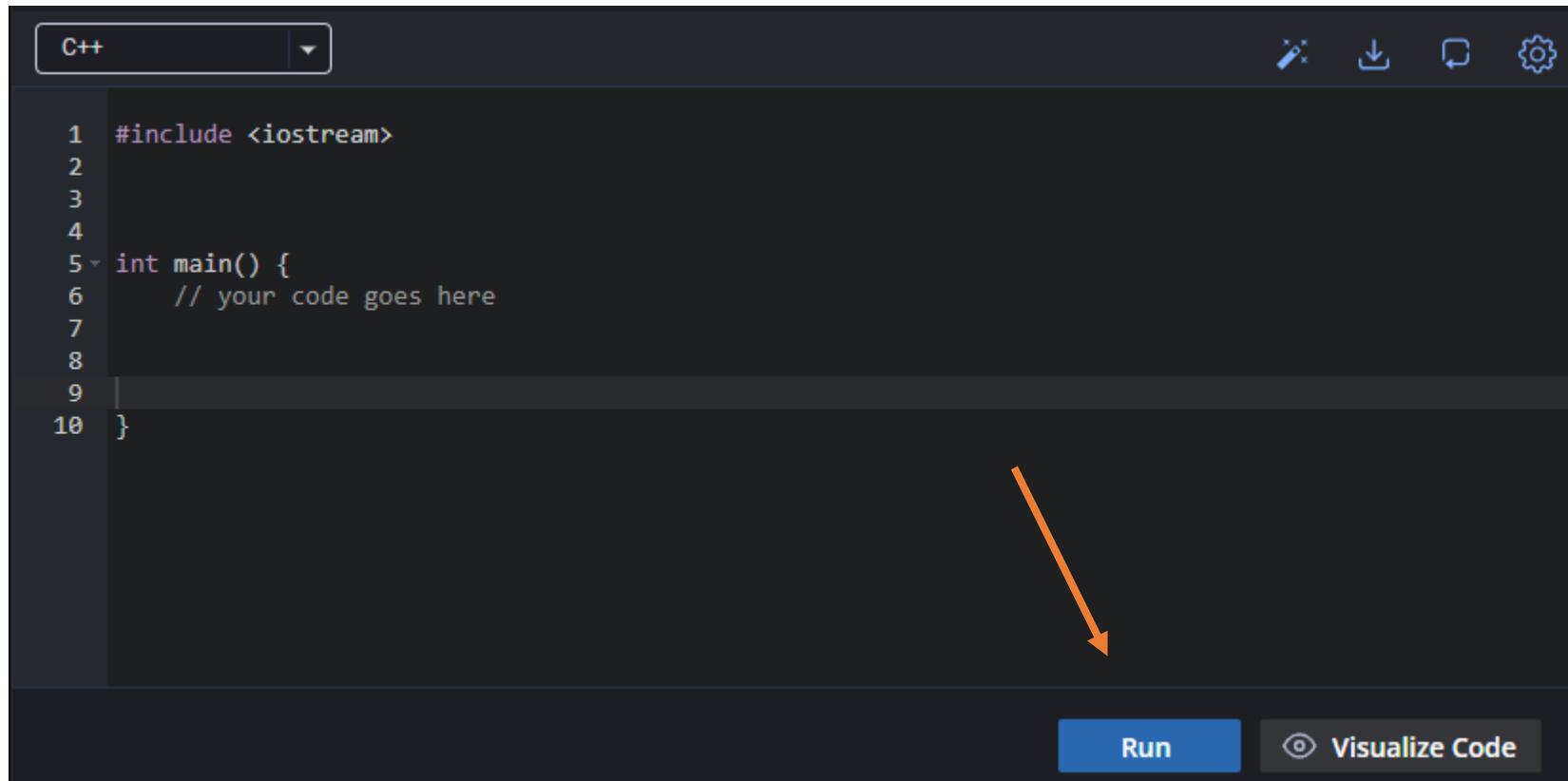
```
int main(){} // однострочный комментарий. От // до конца строки
```

```
int main() {  
    /* многострочный комментарий.  
    Всё, что между символами начала и конца.  
    Не вкладываются друг в друга. */  
}
```

Применяются для документирования кода или чтобы временно отключить его часть без удаления.

Программа просто запускается

Мы не управляем процессом сборки программы и не думаем о нём. По нажатию кнопки происходит магия и программа просто запускается.



Нет ручного управления памятью

Не используем механизмы языка для ручного выделения и освобождения памяти.
Всё, что мы будем создавать, будет удаляться автоматически.

Структура кода в main

```
int main(){
```

Линейная
последовательность

Стейтмент

Стейтмент

Стейтмент

Стейтмент

Стейтмент

Стейтмент

Стейтмент

Базовая единица выполнения программы.
Исполняется и НЕ возвращает значение.

⚠ Кроме стейтментов ещё могут быть
деклараторы (объявления). На текущий
момент мы о них тоже говорить не будем.

```
}
```

Стейтмент (statement)

В русскоязычной литературе можно встретить такие переводы как: оператор, команда, инструкция, и т.д., что не является верным.

Аналогично, как если бы слова «верблюд» не существовало в русском языке, и авторы переводили бы его как «жираф» или «корова», вместо того, чтобы ввести в русский язык слово «верблюд» как есть.

Однако, вышеуказанные переводы настолько «прижились», что встречаются повсеместно в книгах и статьях и все понимают о чём на самом деле идёт речь, так что вполне можно их использовать.



Типичный корован

Стейтмент (statement)

Стейтмент в C++ – это базовая единица **выполнения** программы. Программа на C++ состоит из последовательности стейтментов, которые выполняются по порядку.

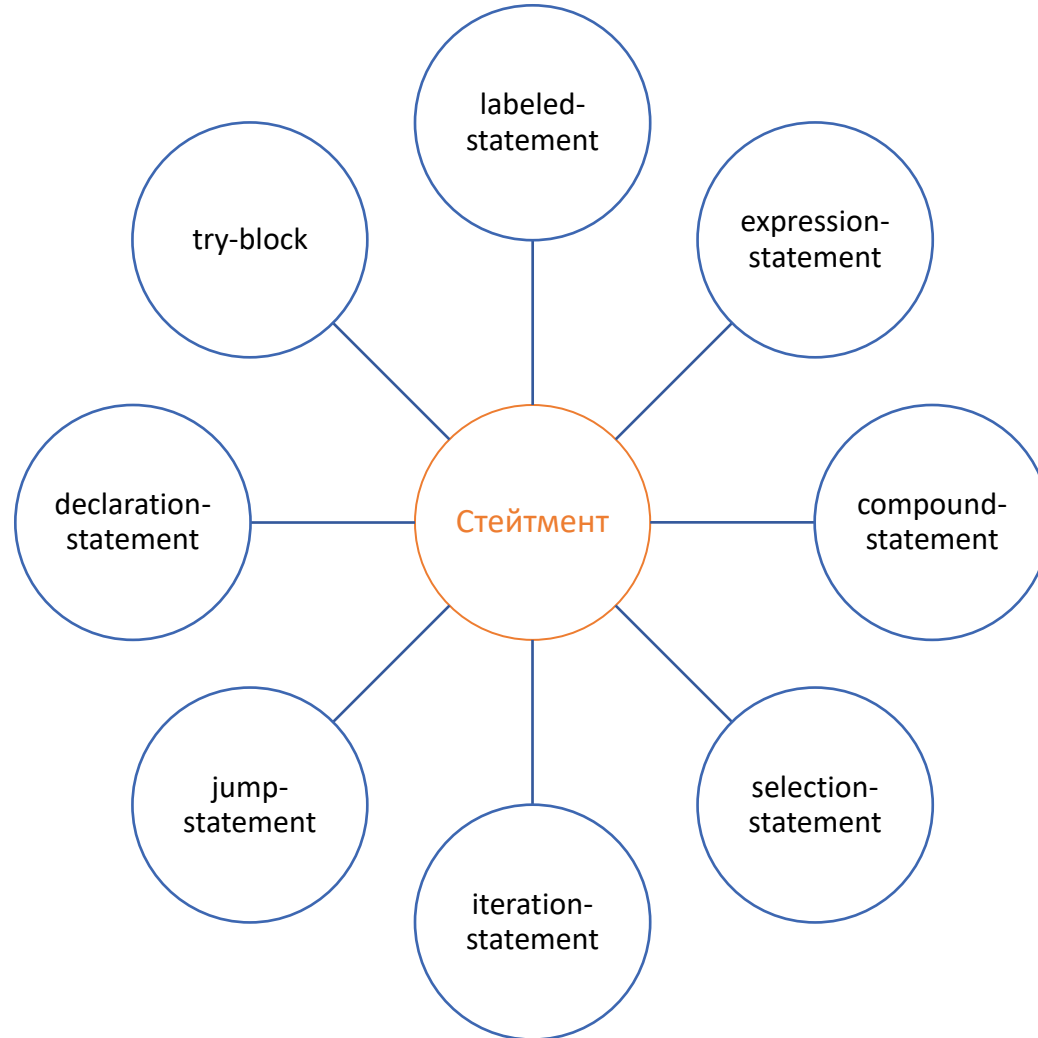
Для стейтментов существует ряд гарантий со стороны языка. Они будут рассмотрены по мере изучения. Например:

```
int a = (1 + print_and_return(2)) + (3 + print_and_return(4));
```

```
int a = (1 + print_and_return(2));  
a += (3 + print_and_return(4));
```

Виды стейтментов

Виды стейтментов



Expression Statement (Стейтмент-выражение)

Любое **выражение**, завершающееся точкой с запятой, становится стейтментом.

выражение ;

Empty Statement (Пустой стейтмент)

Если выражение отсутствует, то получается пустой стейтмент, который не делает ничего. Это не отдельный вид, а частный случай **Expression Statement**.

;

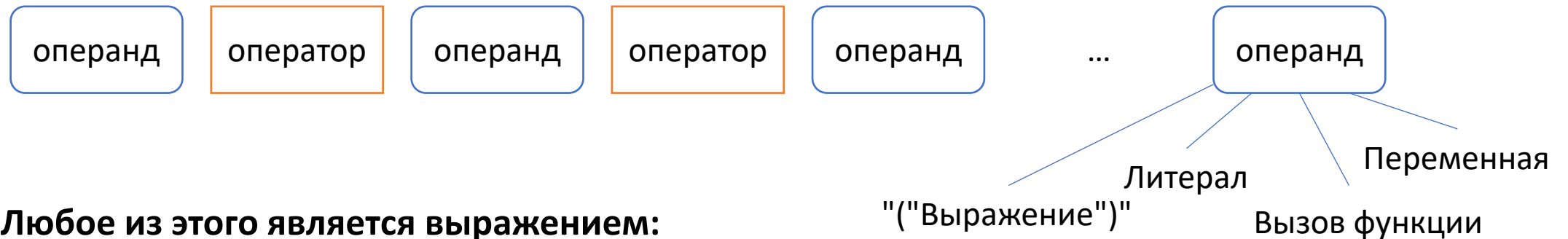
Empty Statement (Пустой стейтмент)

```
int main(){
```

}

Выражение (expression)

Выражение (expression) - последовательность операторов, переменных, литералов и вызовов функций, которая **вычисляется и возвращает значение**.



Любое из этого является выражением:

- Отдельная переменная: x (возвращает своё значение);
- Литерал: 42 (возвращает 42);
- Вызов функции: $\text{sqrt}(25)$ (возвращает 5.0);
- Комбинация всего вышеперечисленного с операторами: $(a + b) * 2 - \text{func}()$.

Литерал

Литерал в C++ — это явное представление в исходном коде **конкретного значения** любого типа данных.

Это константы, которые непосредственно записываются в программе и не могут быть изменены во время выполнения.

```
42          // целочисленный литерал (integer literal)
3.14159     // литерал с плавающей точкой (floating-point literal)
'A'         // символьный литерал (character literal)
"Hello"     // строковый литерал (string literal)
true        // булев литерал (boolean literal)
nullptr     // указатель литерал (pointer literal)
1.2_w       // Пользовательский литерал (user-defined literal)
```

Операторы

Оператор — это заранее определённый символ или ключевое слово в языке, которое выполняет **конкретную операцию** над одним или несколькими операндами и возвращает значение. Порядок выполнения операторов определяется приоритетом и ассоциативностью.

Операторы бывают:

- **Унарные** (работают с одним операндом): $-x$, $++i$, $!flag$, $*ptr$ (разыменование).
- **Бинарные** (работают с двумя операндами): $a + b$, $c = 10$, $x > y$, $n \% 2$.
- **Тернарный** (работает с тремя операндами): единственный такой оператор — условный $a ? b : c$.

Примеры операторов:

$=$, $+$, $-$, $*$, $/$, $\%$, $==$, $!=$, $<$, $>$, $<=$, $>=$, $\&\&$, $||$, $!$, $++$, $--$, $\&$ (адрес или побитовое И), $->$ и т.д.

Приоритет операторов

Приоритет определяет порядок выполнения операторов в выражении. Если выражение содержит несколько операторов, то операторы с более высоким приоритетом выполняются раньше, чем операторы с более низким приоритетом.

Operator Precedence	
$X = 10 + 20 * 30$	Multiply will happen first as * has <i>higher</i> precedence
$X = 10 + 600$	Now addition will happen as + has <i>lower</i> precedence
$X = 610$	We have got the <i>solution</i> for the equation

610

Ассоциативность операторов

Ассоциативность операторов определяет порядок, в котором операнды группируются, когда несколько операторов имеют одинаковый приоритет. Существует два типа ассоциативности:

- Ассоциативность **слева направо** означает, что при наличии в выражении нескольких операторов с одинаковым приоритетом они вычисляются слева направо. Например, в выражении $a + b - c$ сложение и вычитание имеют одинаковый приоритет и являются левоассоциативными, поэтому выражение вычисляется как $(a + b) - c$.
- Ассоциативность **справа налево** означает, что операторы вычисляются справа налево. Например, оператор присваивания = является правоассоциативным. Таким образом, выражение $a = b = 4$ вычисляется как $a = (b = 4)$. Т.е. значение 4 сначала присваивается переменной b , а затем результат этого присваивания (b , которое теперь равно 4) присваивается переменной a .

Ассоциативность операторов

Operator Associativity	
	" / " is evaluated first as it is leftmost
	Now " * " is evaluated.
	We have got the solution for the equation

/ & * both have the same precedence but *left to right (LTR) associativity*

Комплексный пример

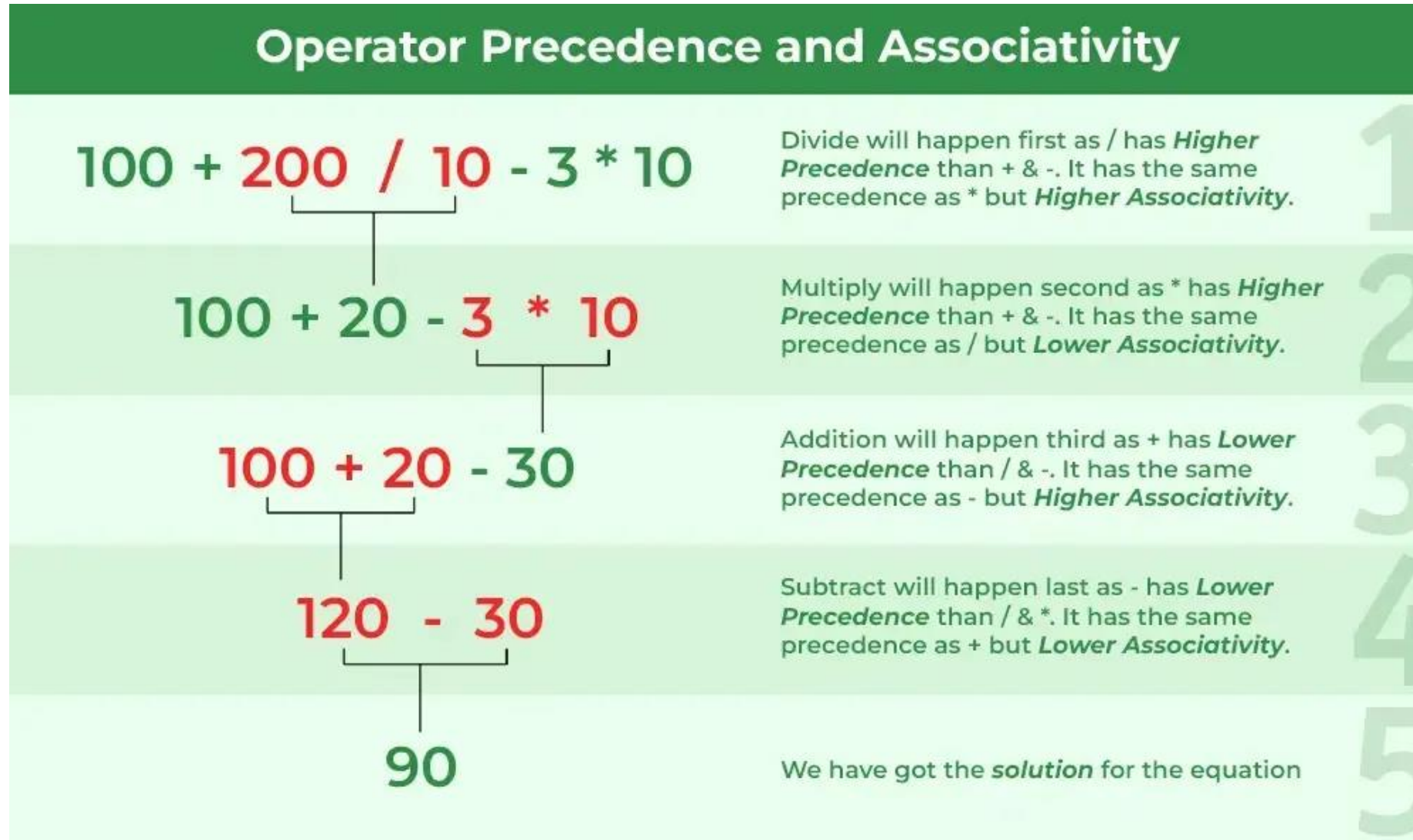


Таблица приоритетов

Полный список операторов, их приоритеты и ассоциативность можно найти в таблице, например: https://en.cppreference.com/w/cpp/language/operator_precedence.html

Precedence	Operator	Description	Associativity
1	<code>a::b</code>	Scope resolution	Left-to-right →
2	<code>a++</code> <code>a--</code> <code>type(a)</code> <code>type{a}</code> <code>a()</code> <code>a[]</code> <code>a.b</code> <code>a->b</code>	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access	
3	<code>++a</code> <code>--a</code> <code>+a</code> <code>-a</code> <code>!a</code> <code>~a</code> <code>(type)a</code> <code>*a</code> <code>&a</code> <code>sizeof</code> <code>co_await</code> <code>new - new[]</code> <code>delete - delete[]</code>	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation	Right-to-left ←
4	<code>a.*b</code> <code>a->*b</code>	Pointer-to-member	Left-to-right →
5	<code>a * b</code> <code>a / b</code> <code>a % b</code>	Multiplication, division, and remainder	
6	<code>a + b</code> <code>a - b</code>	Addition and subtraction	

Expression Statement

```
int main() {  
    ;  
    x;  
    42;  
    x + 42;  
    y * pow(x + 42);  
    z = 1 + y * pow(x + 42);  
    x + y  
}
```

Ошибка. Нет ';' ;
Компилятор будет искать
конец стейтмента и не найдёт

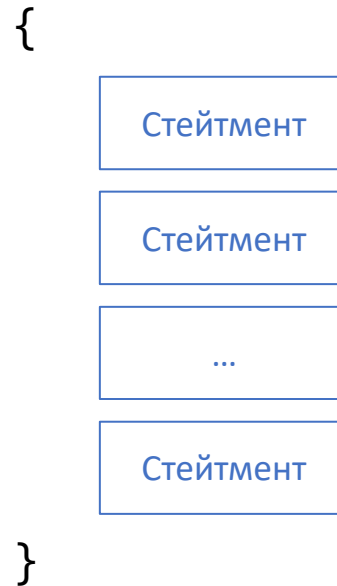
Expression Statement

```
#include <iostream>

int main() {
    std::cout << 2 + 2;
}
```

Compound Statement (Составной стейтмент / Блок)

Составной стейтмент начинается с открывающей фигурной скобки, затем указывается 0 или более стейтментов. Составной стейтмент заканчивается закрывающей фигурной скобкой.



Compound Statement (Составной стейтмент / Блок)

```
{ } // пустой блок. 0 стейтментов внутри
```

```
{ // начало блока
    x + 42; // стейтмент-выражение
    y * pow(x + 42); // стейтмент-выражение
    { // начало вложенного блока
    } // конец вложенного блока
} // конец блока
```

блок – это один стейтмент

Compound Statement (Составной стейтмент / Блок)

Зачем вкладывать стейтменты в блок, если они итак прекрасно исполняются?

У блока есть 2 фишки:

- Внутри блока создаётся отдельная блочная область видимости (Block scope);
- Блок (вместе со всеми вложенными стейтментами считается одним стейтментом), поэтому в конструкция, в которых разрешено указывать только один стейтмент можно использовать блок;

`if` (выражение) стейтмент

слот под один любой стейтмент
можно использовать блок

Compound Statement

```
#include <iostream>
```

```
int main() {  
    { std::cout << 2 + 2; }  
  
    {  
        std::cout << 2 + 2;  
    }  
}
```

Declaration Statement (Стейтмент-объявление)

Declaration Statement вводит в программу новые имена (идентификаторы) и/или влияет на правила работы с именами (например поиска имён).

Без объявлений приходилось бы использовать только встроенные в язык слова и знаки. Нельзя было бы создать переменные, типы и т.д.

Declaration Statement (Стейтмент-объявление)

Формально грамматика declaration-statement в C++20 выглядит так:

```
declaration-statement:  
    block-declaration
```

```
block-declaration:  
    simple-declaration  
    asm-declaration  
    namespace-alias-definition  
    using-declaration  
    using-enum-declaration  
    using-directive  
    static_assert-declaration  
    alias-declaration  
    opaque-enum-declaration
```

using-directive

Один из вариантов declaration-statement это using-directive:

```
using namespace имя_пространства_имён;
```

Например:

```
int main() {  
    // стейтменты  
    using namespace std; // using-directive  
    // стейтменты  
}
```

using-directive

После директивы и до конца текущего блока, в конец обычной процедуры поиска имён добавляется правило: «поискать имя в пространстве имён std»:

```
#include <iostream>

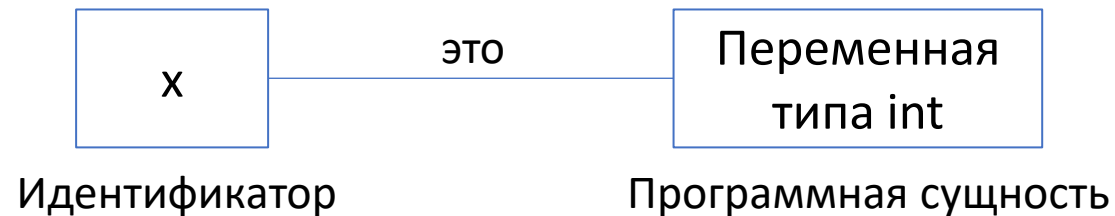
int main(){
    std::cout << 1; // Тут std обязательно
    using namespace std;
    cout << 1;      // Теперь std можно не писать
}
```

simple-declaration

simple-declaration **объявляет** в текущем блоке идентификатор и указывает, что за программная сущность обозначена этим идентификатором: переменная, функция, локальный класс и т.д.

Сейчас мы ограничимся объявлением только переменных/констант.

Объявление - это способ, с помощью которого **имена** вводятся (или повторно вводятся) в программу на C++. После объявления имя можно использовать в программе.



Объявление переменной

Переменная — это именованный блок данных определённого типа. Чтобы объявить переменную, нужно указать её тип и имя. Тип переменной зафиксирован до «смерти» переменной. В общем виде это выглядит так:

```
Type name;
```

```
Type name1, name2, name3;
```

```
Type name1 = value1, name2, name3 = value3;
```

```
auto name = value; // value инициализация
```

```
Type name = value; // Копирующая инициализация
```

```
Type name(value); // Прямая инициализация
```

```
Type name{value}; // Uniform инициализация
```

Объявление переменной - инициализация

Инициализация – это установка начального значения объекта в момент его создания. Инициализатор (если он есть) определяет это значение. Если инициализатора нет, то объект инициализируется значением по умолчанию.

Для примитивных типов, значение по умолчанию не гарантировано:

```
int val = 4;    // val хранит значение 4. 4 - это инициализатор
int val2;      // val2 хранит неопределённый "мусор"
```

Для объектов значение определяется конструктором класса:

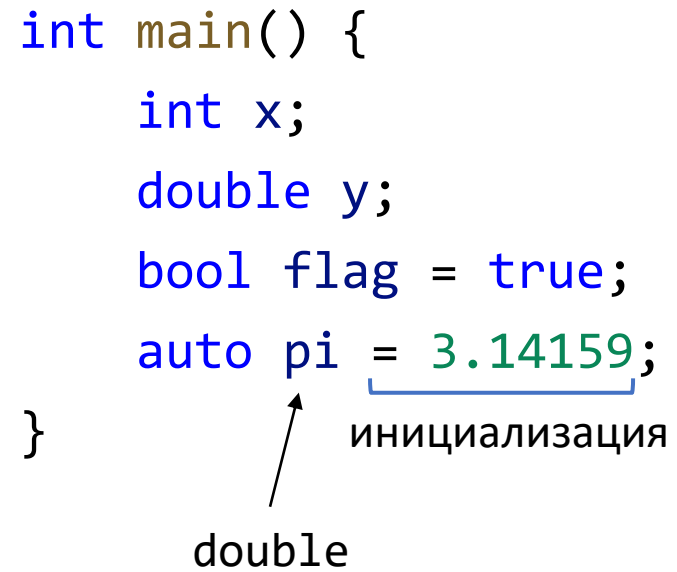
```
string name = "John"; // name хранит значение "John". "John" - это инициализатор
string name2;         // name2 хранит значение ""
```

Объявление переменной

```
int main() {  
    int x;  
    double y;  
    bool flag = true;  
    auto pi = 3.14159;  
}
```

double

инициализация



The diagram illustrates variable declarations and initialization within a C++ function. The code snippet shows a `main` function containing declarations for `int x`, `double y`, `bool flag` (initialized to `true`), and `auto pi` (initialized to `3.14159`). A blue underline is placed under the value `3.14159`, with the Russian word "инициализация" (initialization) written below it. An arrow points from the word "double" at the bottom to the `auto pi` declaration, indicating the variable's type.

Объявление константы

В C++ под понятием константность может скрываться несколько значений. Сейчас мы обсуждаем константы созданные при помощи квалификатора `const`. Стандарт C++ требует, чтобы объект с квалификатором `const` был инициализирован. Однако «инициализация» не обязательно означает явный инициализатор в коде.

Для примитивных типов, инициализатор должен быть обязательно:

```
const int val;           // Ошибка: uninitialized const 'val'  
const int val2 = 1;      // ОК
```

Для объектов значение определяется конструктором класса:

```
const string name = "John"; // ОК. Значение "John"  
const string name2;         // ОК. Значение ""
```

Объявление константы

После создания, значение константы не может быть изменено:

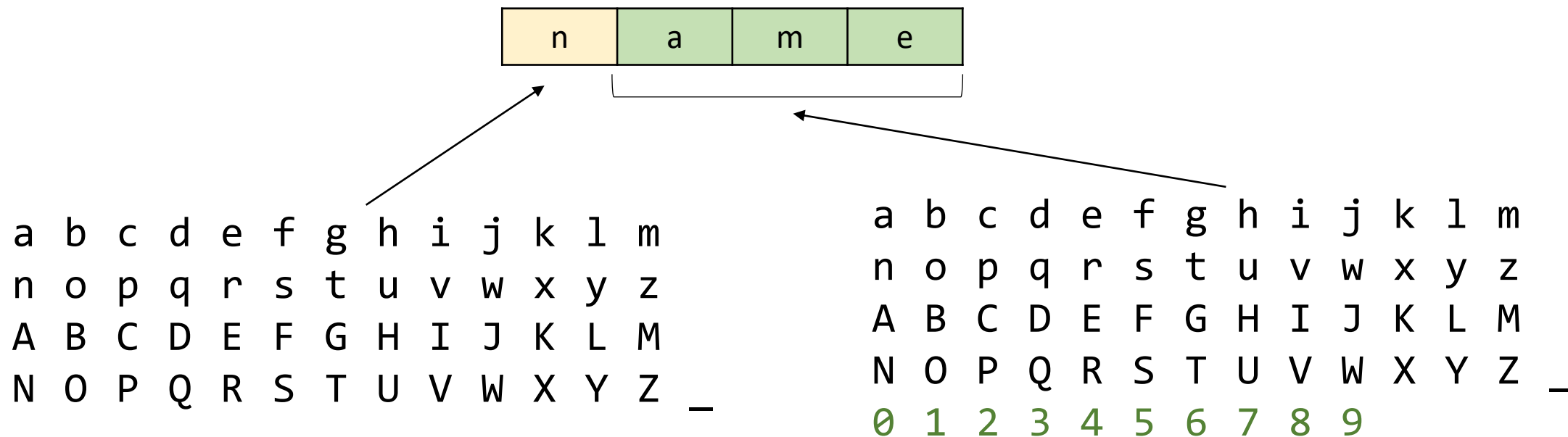
```
int x = 1;  
const int y = 1;  
x = 100;    // ОК  
y = 100;    // Ошибка
```

Можно и так:

```
int x;      // Мусор  
cin >> x;   // Ввод значения с консоли  
const int y = 2 + x;
```

Идентификатор (Identifier)

Идентификатор (имя) - это последовательность букв и цифр произвольной длины.



Идентификатор

```
int main() {  
    int _;           // OK  
    int _42;         // OK  
    int 4cheeses;    // не OK  
  
    int myname;  
    int myName;      // camel case  
    int MyName;      // pascal case  
    int my_name;     // snake case  
  
    int Число😁 = 5;  // OK  
}
```

Поиск имени (Name lookup)

Если компилятор встречается в тексте программы идентификатор, то он выполняет процедуру поиска **объявления**, которое сообщит ему, с какой программной сущностью идентификатор связан в текущей точке программы. В простом варианте:

1. Ищи объявление в текущем блоке { ... } вверх от текущей позиции.
2. Если не нашёл – ищи в объемлющем блоке (тоже вверх по коду).
3. Повторяй, пока не дойдёшь до глобальной области.
4. Если нашёл – СТОП. Выше не смотри, даже если найденное имя не подходит по типу или контексту.
5. Если не нашёл и в процессе поиска встречались директивы using смотри в указанных пространствах имён.
6. Если нашёл в нескольких пространствах имён - то ошибка неоднозначности (ambiguous).
7. Если не нашёл нигде – ошибка компиляции (**имя was not declared**).

Scope (Область видимости)

```
int main() {  
    x + 42;    // Ошибка. Имя x не объявлено  
    int x = 1;  
    x + 42;    // Ок. Имя x объявлено  
}
```

Scope (Область видимости)

Вышесказанное означает, что компилятор "видит" идентификатор начиная с определённой точки кода (Point of declaration).

Часть кода, в которой компилятор связывает идентификатор с конкретной программной сущностью называется **областью видимости** этого объявления/идентификатора.

```
int main() {  
    int x;  
    x + 42;  
}
```

point of declaration

 Область видимости идентификатора x

 Идентификатор x не виден

Scope

```
int main() {  
    int x;  
    x + 42;  
}
```

```
int main() {  
    int x = 1;  
    x + 42;  
}
```

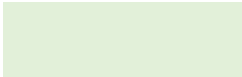
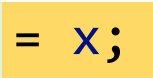
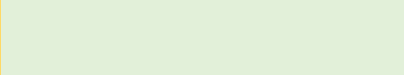
```
int main() {  
    int x = x;  
    int y(y);  
    int z{z};  
    int k[k];  
}
```

 Область видимости

Повторное определение имени в одной области видимости

```
int main() {  
    int a;  // Объявляем и определяем a  
    int a;  // Не ОК нарушение ODR  
  
    double a; // Не ОК нарушение ODR  
}
```

Вложенные блоки

```
int main() {  
    int x;   
    {  
        int y = x;   
    }   
}
```



Область видимости идентификатора x



Область видимости идентификатора x и y

Вложенные блоки

```
#include <iostream>
```

Потенциальная область
видимости идентификатора

```
int main() {  
    int x = 1;  
    {  
        int x = 2;  
        std::cout << x; // Вывод: 2  
    }  
}
```

shadowing

Область видимости идентификатора **x**

Область видимости **вложенного** идентификатора **x**

Вложенные блоки

```
int x;  
int y;  
{  
    int x;  
}
```

Идентификатор	Тип	Адрес
x	int	0x7ffe6625f9d4

Идентификатор	Тип	Адрес
y	int	0x7ffe6625f9d8
x	int	0x7ffe6625f9d4

Идентификатор	Тип	Адрес
x	int	0x7ffe6625f9dc
y	int	0x7ffe6625f9d8
x	int	0x7ffe6625f9d4

Идентификатор	Тип	Адрес
y	int	0x7ffe6625f9d8
x	int	0x7ffe6625f9d4

Время жизни (Storage duration)

Локальные переменные/константы, без доп. квалификаторов, обладают автоматическим временем жизни (automatic storage duration). Они создаются в точке объявления и уничтожаются при выходе из области видимости. Если идентификатор не вышел из области видимости, а был скрыт (shadowing) объект не уничтожается.

```
#include <iostream>

int main() {
    int x = 1; // Выделяем память под x
    {
        int y = 2; // Выделяем память под y
        std::cout << y;
    } // Освобождаем память занятую y
} // Освобождаем память занятую x
```

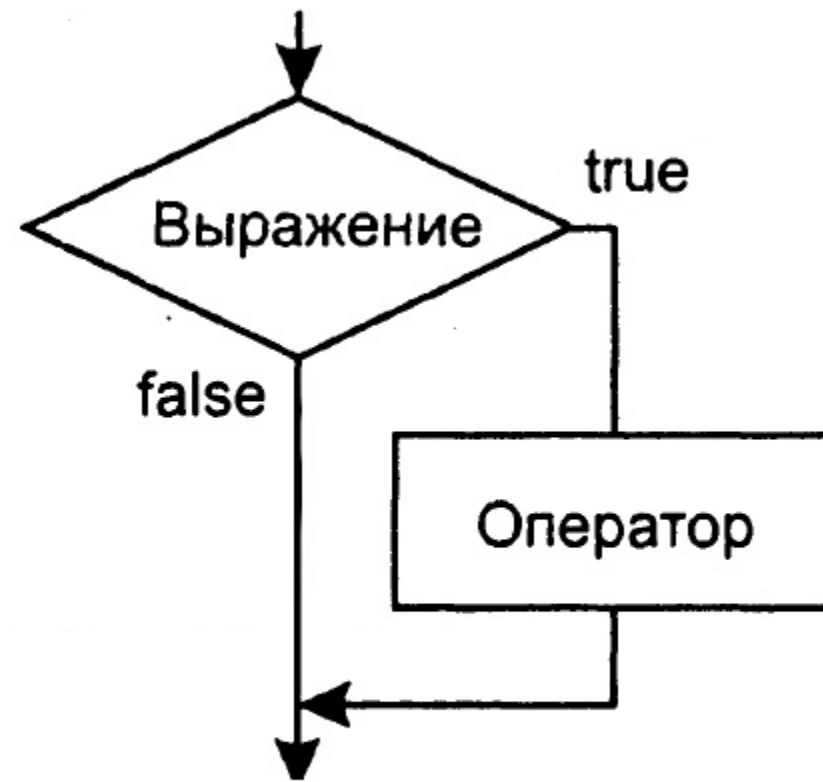
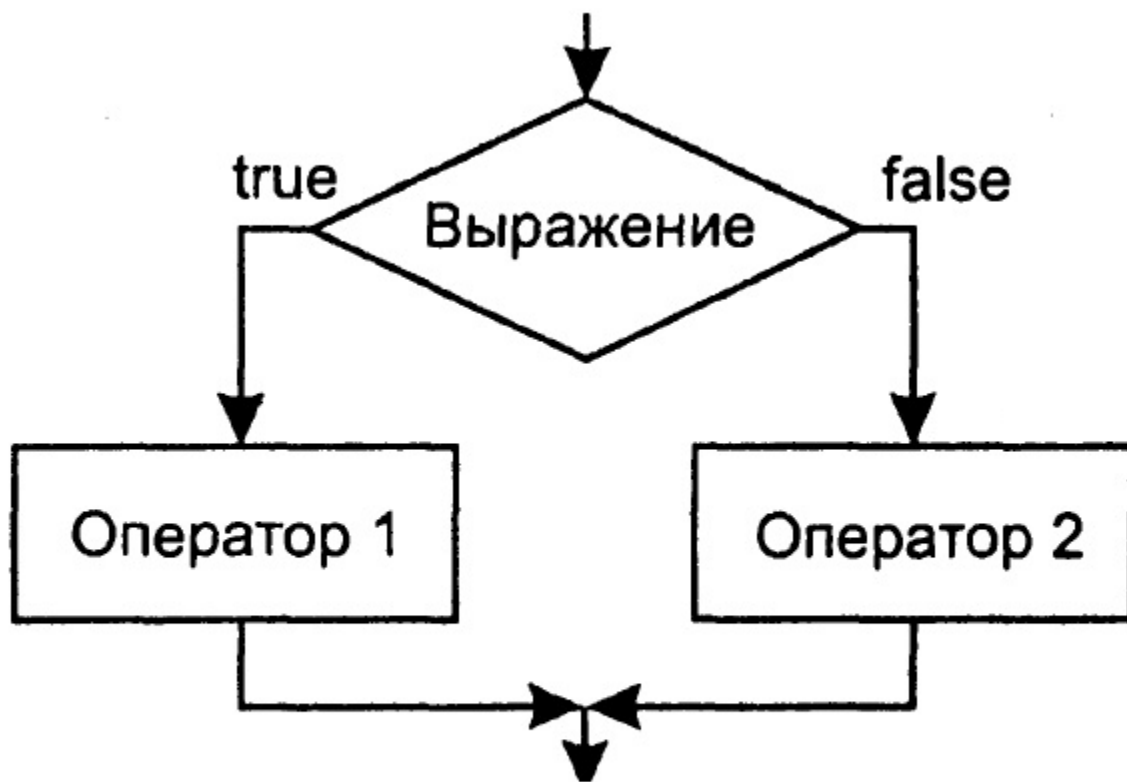
Selection Statements (Стейтменты выбора)

В стандарте указано 2 вида:

- if statement – условный оператор if;
- switch statement – оператор множественного выбора switch.

Внимание! На самом деле if и switch – не операторы, а стейтменты. Некорректный перевод на русский настолько закрепился в языке, что стал почти каноном.

Условный оператор



Структурная схема условного оператора

Тип bool

```
bool a = true;    // Истина  
bool b = false;   // Ложь
```

* Маленькими буквами

Преобразование типа bool

Преобразование **bool** в числовые типы:

- Значение **true** преобразуется в **1** для типов **int**, **float**, **double** и других числовых типов;
- Значение **false** преобразуется в **0** для типов **int**, **float**, **double** и других числовых типов;

Преобразование числовых типов в **bool**:

- Любое НЕнулевое значение числового типа преобразуется в **true**;
- Значение **0** числового типа преобразуется в **false**;

* Преобразование происходит автоматически

Преобразование типа bool

```
#include <iostream>

int main() {
    int num = true; // Преобразование bool в int, значение num будет равно 1
    float f = false; // Преобразование bool в float, значение f будет равно 0

    int num2 = 0;
    bool b2 = num2; // Преобразование int в bool, значение b2 будет равно false

    int num3 = -70; // Преобразование int в bool, значение b2 будет равно true
    bool b3 = num3; // Преобразование int в bool, значение b2 будет равно false

    std::cout << "int: " << num << std::endl;
    std::cout << "float: " << f << std::endl;
    // По умолчанию тип bool выводится на экран как 1 и 0
    // std::boolalpha меняет способ вывода на true и false
    std::cout << "bool: " << std::boolalpha << b2 << std::endl;
    std::cout << "bool: " << std::boolalpha << b3 << std::endl;
}
```

Откуда берётся true и false

Операторы сравнения

Оператор	Проверяет на	Пример использования	Результат
==	Равенство	3 == 7	false
!=	Неравенство	3 != 7	true
<	Меньше	3 < 7	true
>	Больше	3 > 7	false
<=	Меньше или равно	7 <= 7	true
>=	Больше или равно	7 >= 7	true

Начиная с C++ 20 был добавлен оператор трёхстороннего сравнения <=> ("spaceship" космический корабль). Этот оператор возвращает не тип bool, поэтому здесь не будем его рассматривать.

Логические операторы

Название	Как выглядит	Как использовать
И	<code>&&</code> <code>and</code>	<code>a && b;</code> <code>a and b;</code>
ИЛИ	<code> </code> <code>or</code>	<code>a b;</code> <code>a or b;</code>
НЕ	<code>!</code> <code>not</code>	<code>!a;</code> <code>not a;</code>

a	b	a and b	a or b	not a
false	false	false	false	true
true	false	false	true	false
false	true	false	true	true
true	true	true	true	false

Логические операторы применяются только к операндам типа `bool`, поэтому перед их применением будет попытка преобразовать операнды в `bool`. Если это не возможно, то получаем ошибку.

Операторы И и ИЛИ вычисляются по сокращённым правилам: если результат можно получить вычислив первый аргумент, второй не вычисляется:

```
false && std::cout << 1; // пусто
```

```
true && std::cout << 2; // 2
```

Операторы сравнения. Возможные ошибки

```
int a = 10;  
-1 < a < 2    // true
```

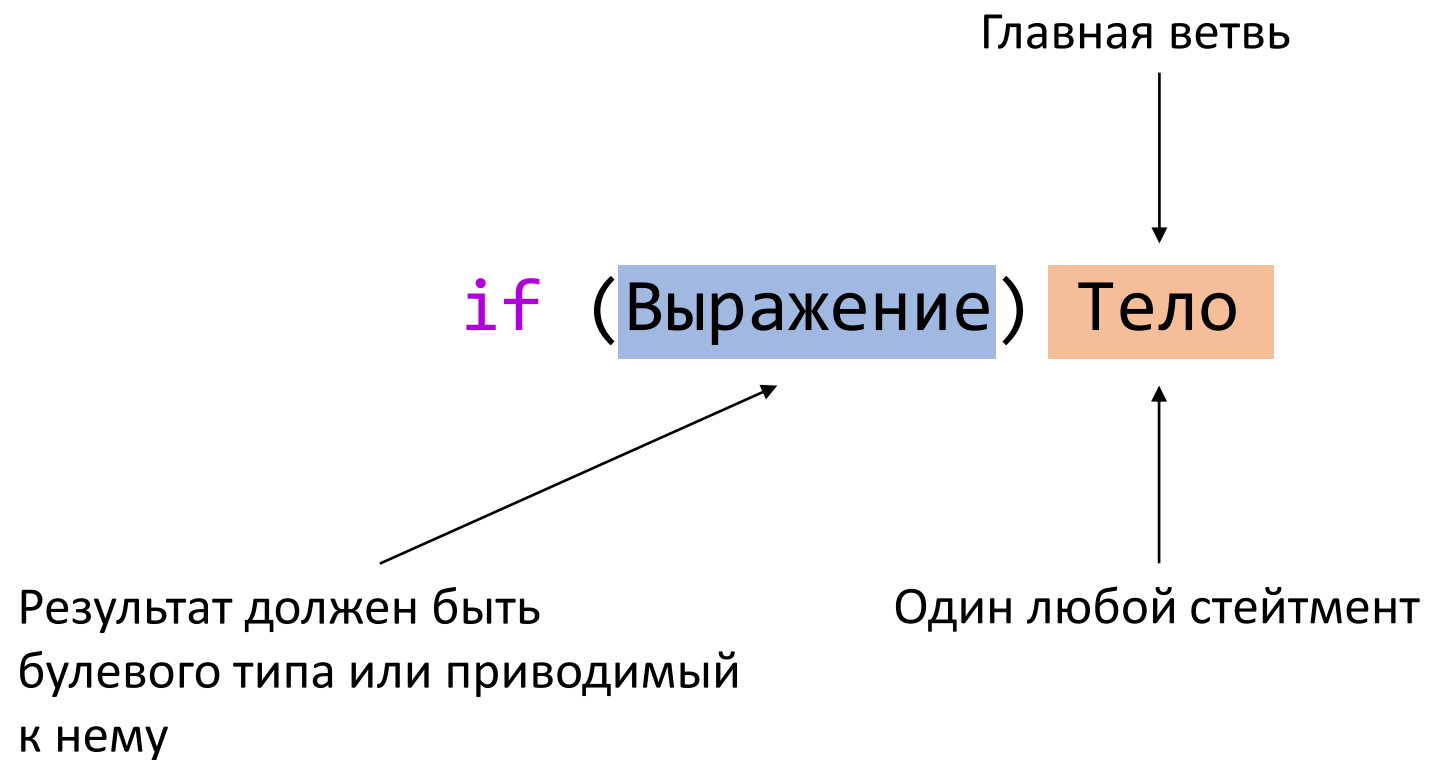
Последнее выражение вычисляется последовательно:

```
    -1 < a < 2  
(-1 < a) < 2  
    true < 2  
        true
```

Если хотите получить результат по математическим правилам пишите:

```
(-1 < a) and (a < 2)    // false
```

if



if

Ключевое слово **если**

Выражение

Тело

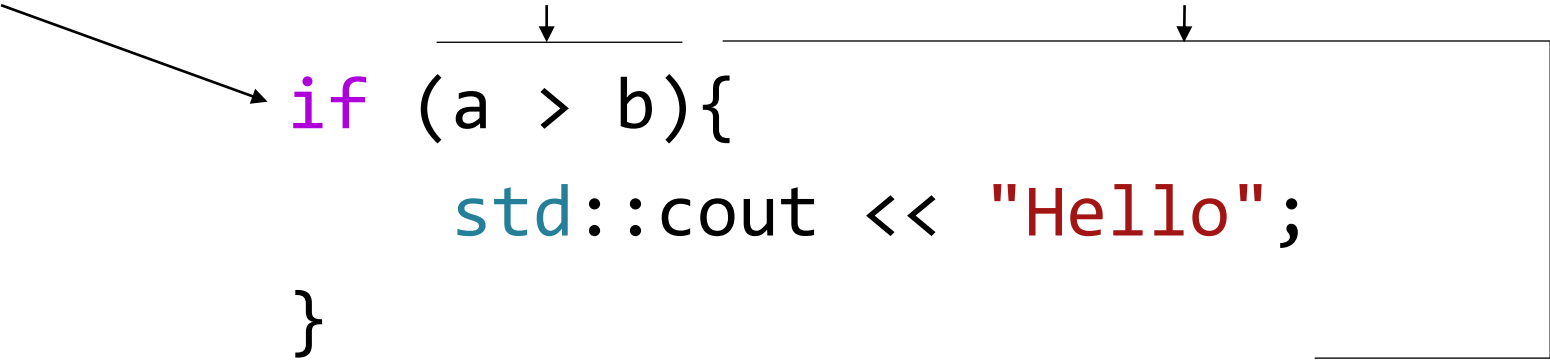


```
if (a > b) std::cout << "Hello";
```

Ключевое слово **если**

Выражение

Тело

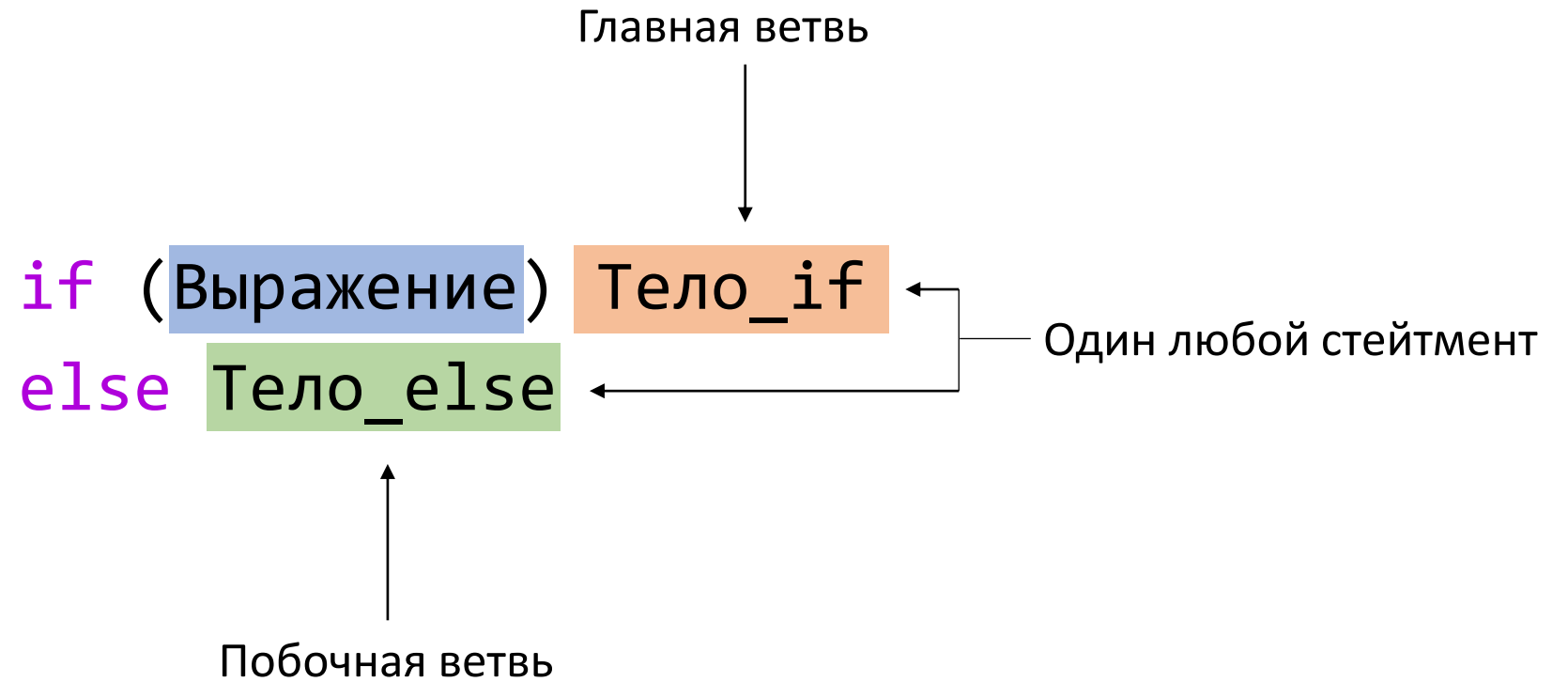


```
if (a > b){  
    std::cout << "Hello";  
}
```

if(инициализация; проверка)

init-statement
↓
if (int res = a > b; res) std::cout << "Hello";
else std::cout << "Bye";

if-else



if-else

Выражение



Ключевое слово **если** → **if** (a > b) **std::cout** << "Hello"; ← Тело **if**

Ключевое слово **иначе** → **else** **std::cout** << "Bye"; ← Тело **else**

Выражение



Ключевое слово **если** → **if** (a > b){
 std::cout << "Hello"; ← Тело **if**

Ключевое слово **иначе** → } **else** {
 std::cout << "Bye"; ← Тело **else**
}

if и область видимости

```
int main() {  
    if (1) int a = 1;  
  
    cout << a;  
}
```

```
int main() {  
    {  
        if (1) int a = 1;  
    }  
    cout << a;  
}
```

```
int main() {  
    {  
        int a = 1;  
    }  
    cout << a;  
}
```

if и область видимости

```
int main() {  
    if (?) int a = 1;  
    else int a = 2;  
  
    cout << a;  
}
```

главная

```
int main() {  
    {  
        int a = 1;  
    }  
    cout << a;  
}
```

побочная

```
int main() {  
    {  
        int a = 2;  
    }  
    cout << a;  
}
```

if и область видимости

```
int main() {  
    if (int a = 1; a)  
        int a = 1;  
  
    cout << a;  
}
```

```
int main() {  
    {  
        if (int a = 1; a)  
            int a = 1;  
    }  
    cout << a;  
}
```

```
int main() {  
    {  
        int a = 1;  
        int a = 1; // He OK  
    }  
    cout << a;  
}
```

if и область видимости

```
int main() {  
    if (int a = 1; a){  
        int a = 1; // Не OK  
    }  
  
    cout << a;  
}
```

```
int main() {  
    if (int a = 1; a){  
        {  
            int a = 1 ; // OK  
        }  
    }  
  
    cout << a;  
}
```

if

```
if (/* условие */)
{
    /* true */
}
```

If-else

```
if (/* условие */)
{
    /* true */
}
else
{
    /* false */
}
```

If-else if

```
if (/* условие 1 */)
{
    /* true */
}
else if (/* условие 2 */)
{
    /* true */
}
else
{
    /* false */
}
```

Вложенный if

```
if (/* условие */)
{
    if (/* условие */)
    {
        /* true */
    }
    else
    {
        /* false */
    }
}
else
{
    if (/* условие */)
    {
        /* true */
    }
    else
    {
        /* false */
    }
}
```

If-else (ошибки)

```
if (a > b);
```

```
if (a > b){/* код */};  
else {/* код */}
```

```
if (a > b)  
    if (a > c) std::cout << "Hello";  
else std::cout << "Bye";
```

If (другие формы)

Существуют ещё две формы `if` не связанные с потоком **исполнения** программы (изучим позже):

- `if constexpr` (C++17) – это инструкция выбора, которая отбрасывает одну из ветвей на этапе компиляции. Условие должно быть константным выражением типа `bool`. Та ветвь, условие которой ложно, не инстанцируется и не компилируется (в контексте шаблонов).
- `if constexpr` (C++23) – это инструкция, которая проверяет, вычисляется ли текущий контекст как «явно константный». Это позволяет одной и той же `constexpr`-функции использовать разные алгоритмы в зависимости от того, вызывается ли она во время компиляции или во время выполнения.

Тернарный оператор (?:)

```
variable = a > b ? a : b;
```

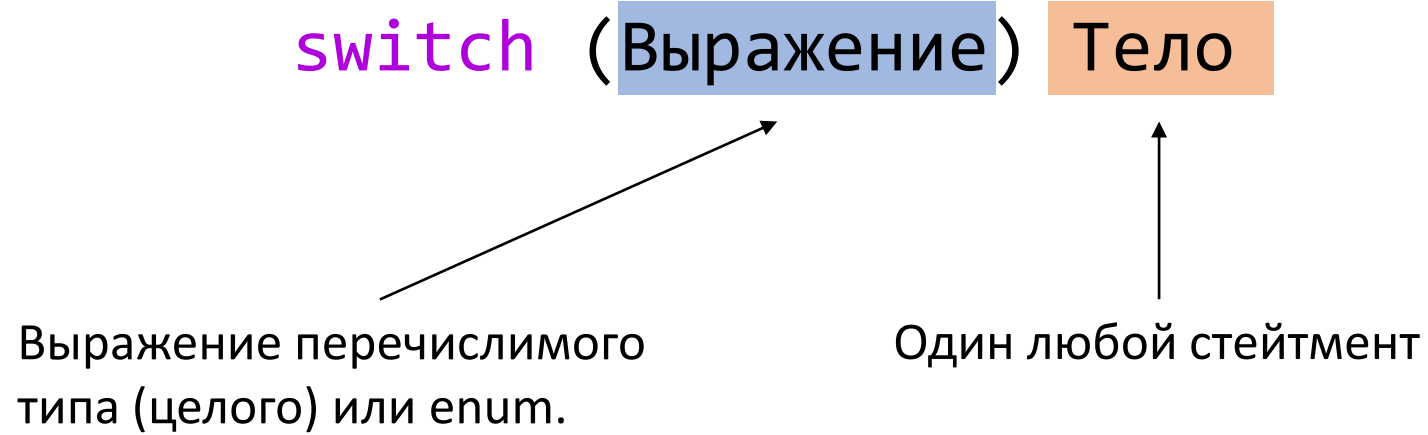
```
/* результат */ = /* условие */ ? /* выражение 1 */ : /* выражение 2 */;
```

* выражение 1 и выражение 2 должны быть одного или приводимого к одному типу

switch

switch (Выражение) Тело

Выражение перечислимого
типа (целого) или enum.



Один любой стейтмент

switch

```
int main() {  
    switch (1) ;  
}
```

```
int main() {  
    switch (1) {  
    }  
}
```

```
int main() {  
    switch (1) {  
        cout << "Hello ";  
        cout << "world";  
        cout << "!";  
    }  
}
```

Labeled statement (Метка)

Стандартом определены 3 типа меток:

- метка для `goto`. Будет рассмотрена далее;
- `case`.
- `default`.

Метки `case` и `default` могут быть использованы только внутри `switch`. При совпадении (`==`) выражения в скобках у `switch` и значения метки `case` поток исполнения перепрыгивает на эту метку пропуская весь код до неё и продолжает исполнение команд **до конца** `switch` игнорируя все остальные метки.

В случае отсутствия совпадения с `case`, используется метка `default`, которая действует аналогично `case`. Метка `default` может быть расположена в любом месте `switch`.

Если внутри `switch` создаются переменные, то не должно быть метки, которая перепрыгивает создание переменной, иначе компилятор сообщит об ошибке.

switch

Выражение
↓
`switch(value){`
 `case 1:` ←
 `std::cout << "one";`
 `case 2:` ←
 `std::cout << "two";`
 `default:` ←
 `std::cout << "zero";`
}

Метка

Jump Statement break (Стейтмент перехода break)

Т.к. после перехода на метку `case` или `default` исполнение кода продолжается до конца `switch`, иногда есть необходимость прервать исполнение кода и выйти из `switch` досрочно. Для этих целей существует стейтмент перехода `break`; Как только он встречается в коде, поток исполнения выходит за пределы текущего блока.

`break` разрешено использовать только в `switch` и в циклах.

В C++ есть только `break` без метки.

switch

Выражение
↓

```
switch(value){  
    case 1:  
        std::cout << "one";  
        break;  
    case 2:  
        std::cout << "two";  
        break;  
    default:  
        std::cout << "zero";  
}
```



switch(инициализация; выражение)

init-statement
↓

```
switch (int res = a > b; res ){  
    case true: std::cout << " :)"; break;  
    default: std::cout << " :(";  
}
```

switch и области видимости

Всё, что было сказано про области видимости в отношении if, справедливо и здесь.

Стоит упомянуть, что case и default - это просто метки, поэтому они не создают свои области видимости. Если нужно, используйте Блоки.

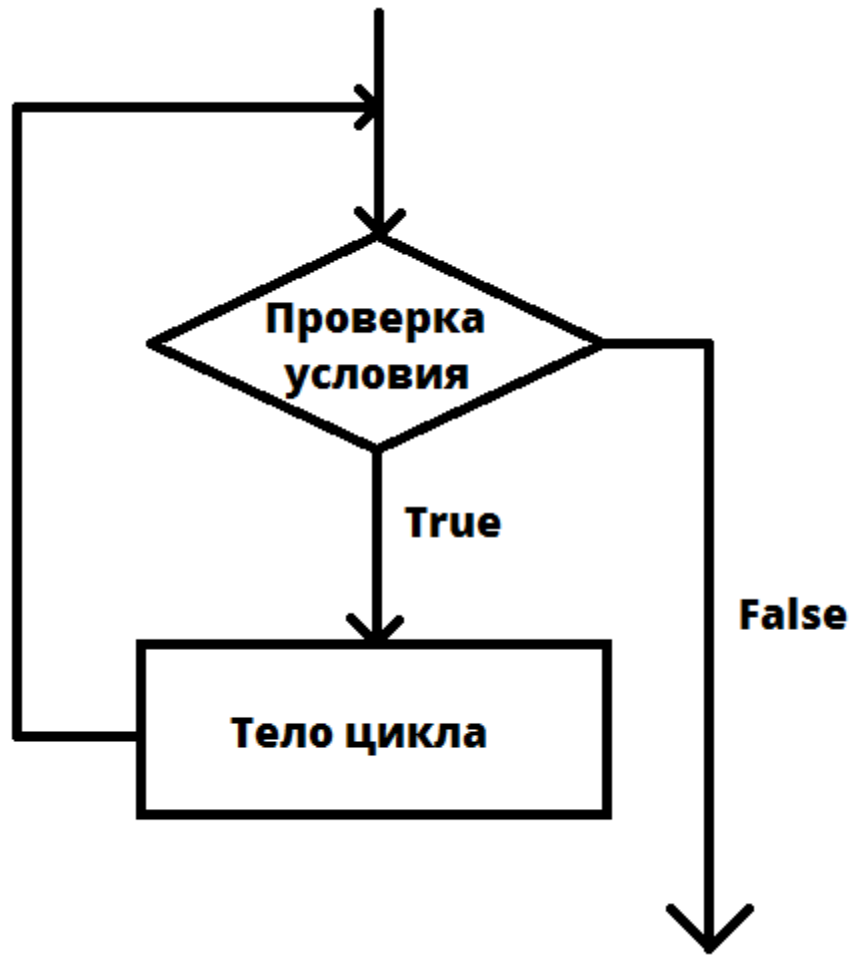
Iteration Statements (Циклы)

В стандарте указано 4 вида:

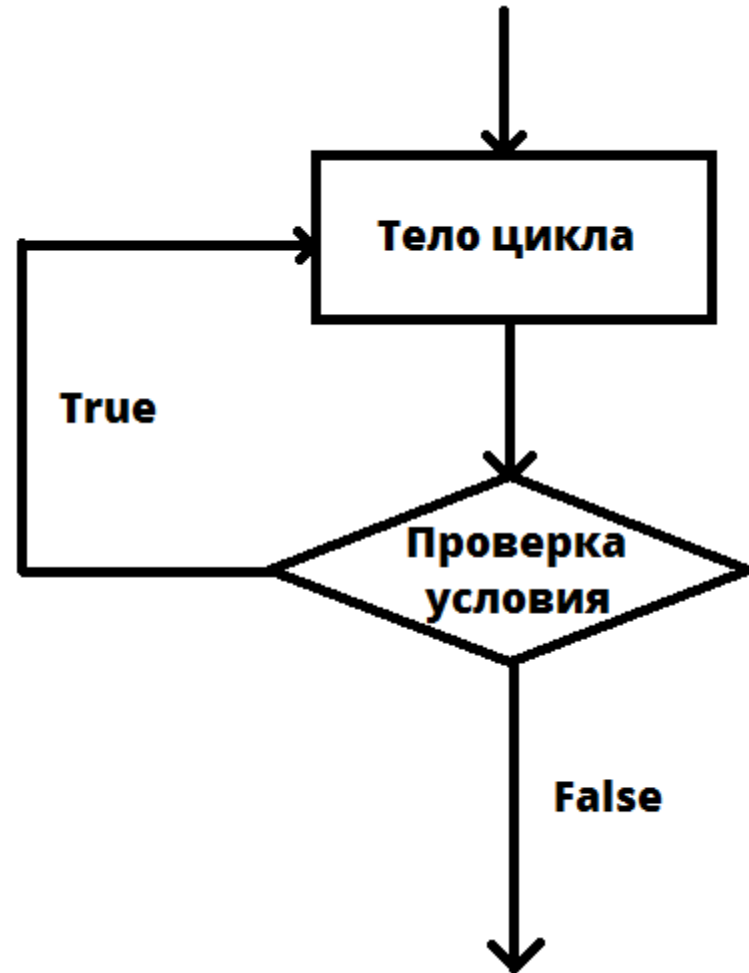
- `while`. Цикл с предусловием;
- `do-while`. Цикл с постусловием;
- `for`. Цикл со счётчиком;
- Range-based `for` (с C++11). `for` на основе диапазона.

Внимание! Циклы тоже иногда называют операторами, но это стейтменты. Та же история, что и с `if`

Оператор цикла



while



do-while

while

`while` (Выражение) Тело

Результат должен быть
булевого типа или приводимый
к нему

Один любой стейтмент

while

Ключевое слово

Выражение

Тело

```
while (a > b) std::cin >> a;
```

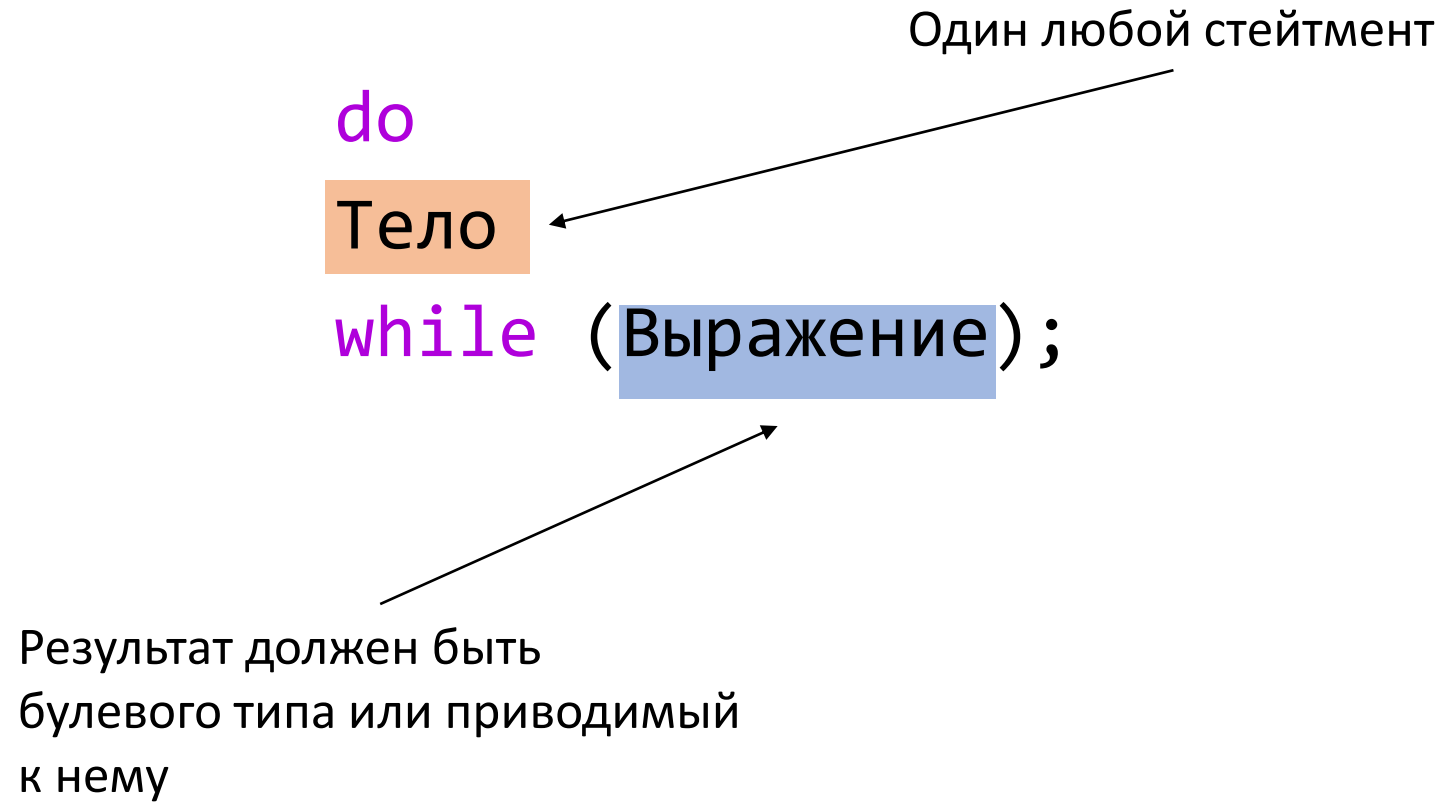
Ключевое слово

Выражение

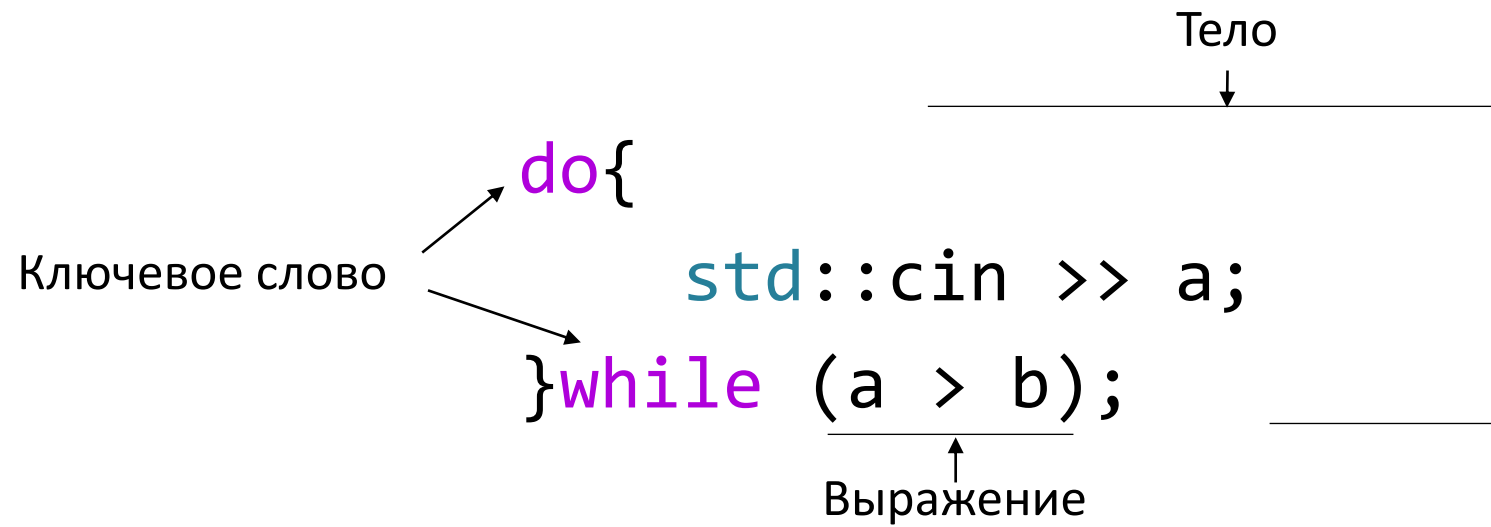
Тело

```
while (a > b){
    std::cin >> a;
}
```

do-while



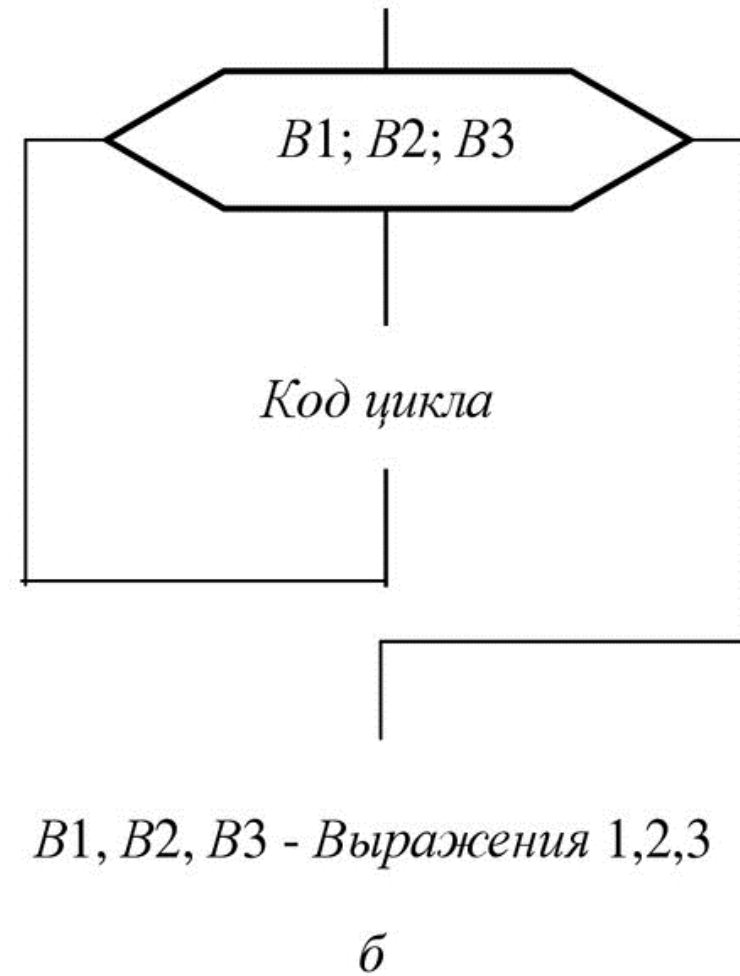
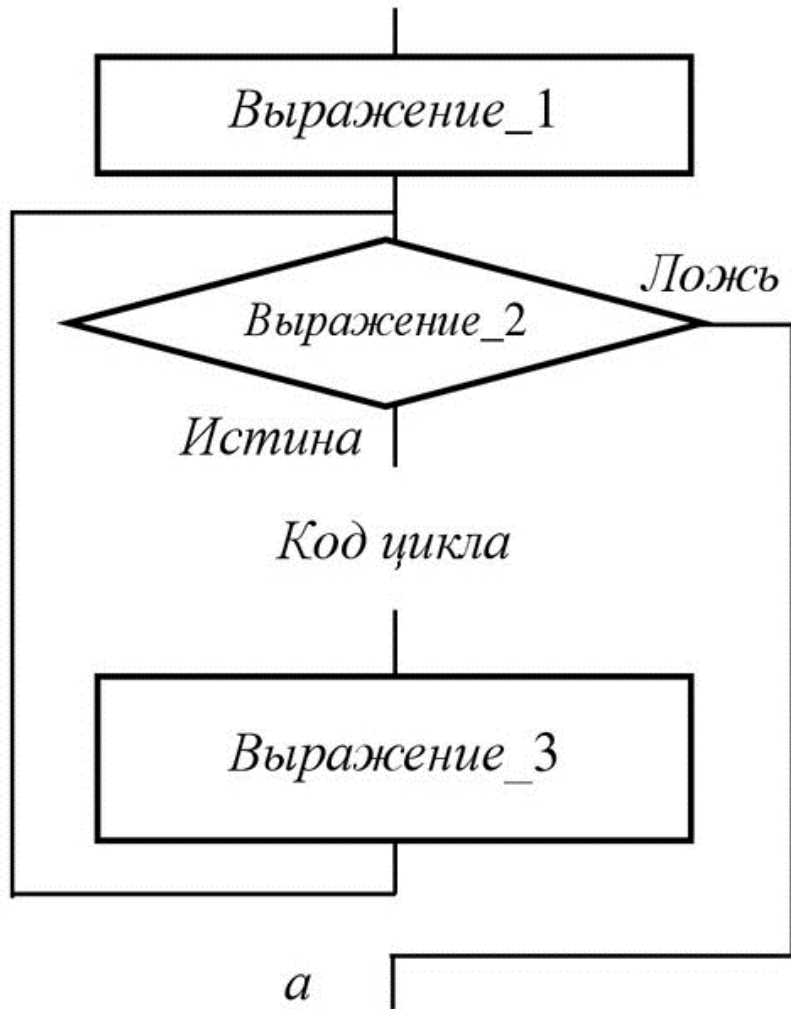
do-while



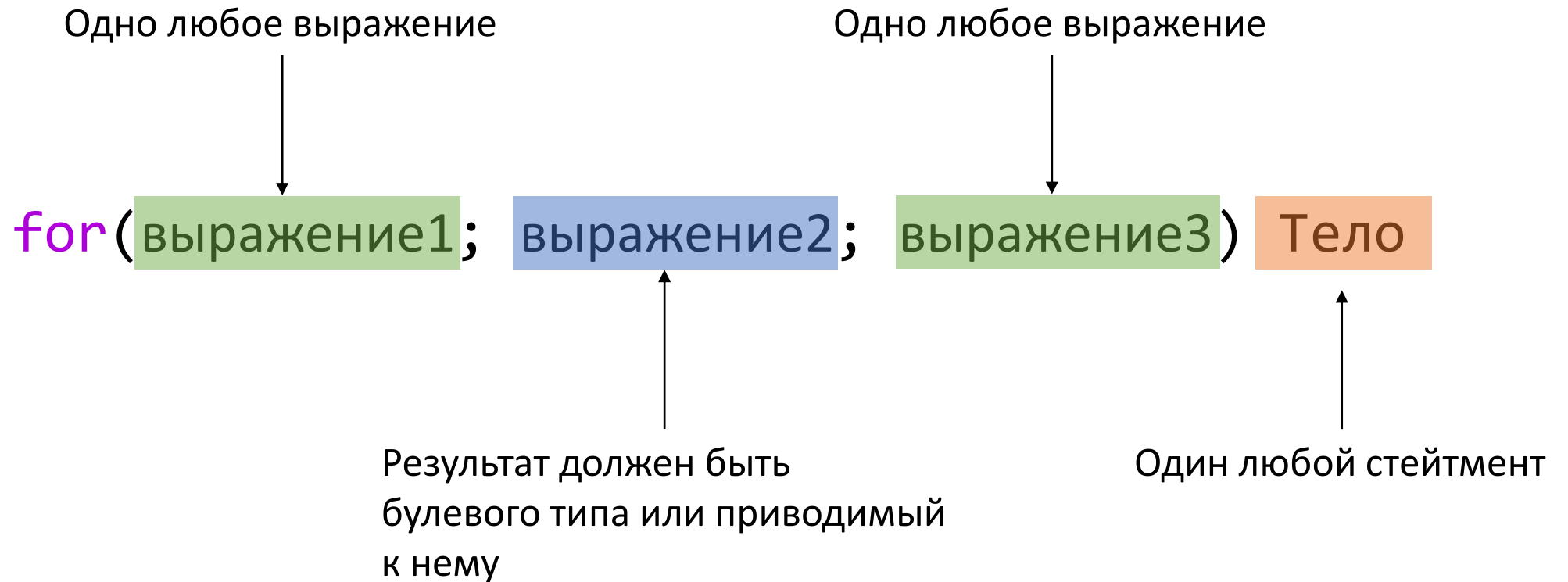
do-while и область видимости

```
int main() {  
    do  
        int a = 1;  
    while (a); // а не существует  
}
```

Оператор цикла for



for



for (выражение1; выражение2; выражение3)

Выражение1 – любое выражение или инициализация переменной. Обычно - инициализация переменной счётчика или нескольких;

Выражение2 – любое выражение или инициализация переменной. Обычно - выражение проверяющее условие работы цикла. Если выражение не указано, то считается, что оно равно true.

Выражение3 – выражение. Обычно инкремент/декремент счётчика(ов).

* каждое из выражение не обязательное (можно не писать), но точки с запятой писать нужно.

for

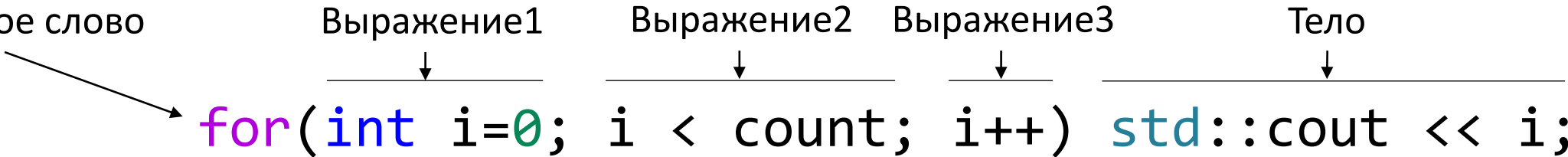
Ключевое слово

Выражение1

Выражение2

Выражение3

Тело



The diagram shows a C++ for loop: `for(int i=0; i < count; i++) std::cout << i;`. Above the code, four labels are positioned: 'Ключевое слово' (with an arrow pointing to 'for'), 'Выражение1' (with an arrow pointing to 'int i=0;'), 'Выражение2' (with an arrow pointing to 'i < count;'), 'Выражение3' (with an arrow pointing to 'i++'), and 'Тело' (with an arrow pointing to 'std::cout << i;'). Each label is underlined.

```
for(int i=0; i < count; i++) std::cout << i;
```

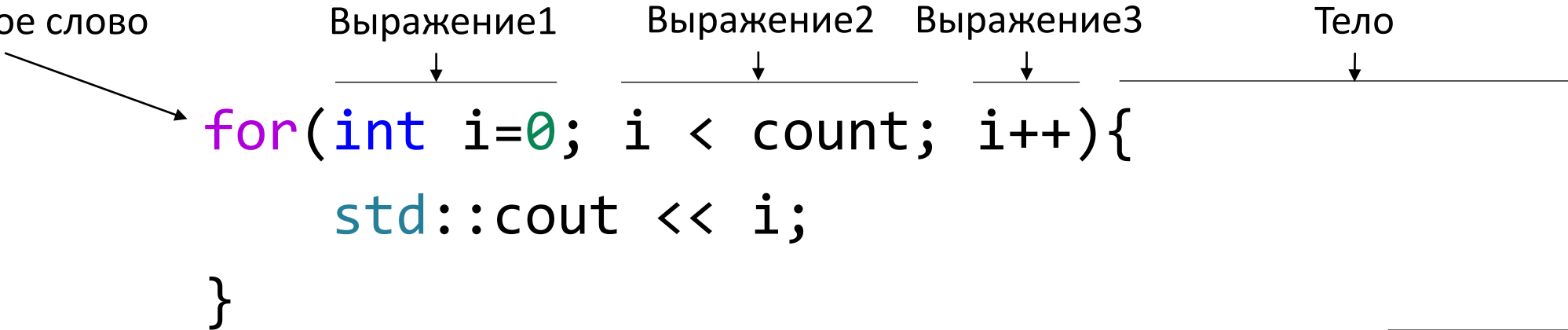
Ключевое слово

Выражение1

Выражение2

Выражение3

Тело



The diagram shows a C++ for loop with a block body: `for(int i=0; i < count; i++){ std::cout << i; }`. Above the code, four labels are positioned: 'Ключевое слово' (with an arrow pointing to 'for'), 'Выражение1' (with an arrow pointing to 'int i=0;'), 'Выражение2' (with an arrow pointing to 'i < count;'), and 'Выражение3' (with an arrow pointing to 'i++'). Each label is underlined. The body of the loop, 'std::cout << i;', is enclosed in curly braces. A line from the 'Тело' label points to the opening brace, and a line from the closing brace extends to the right.

```
for(int i=0; i < count; i++){  
    std::cout << i;  
}
```

range-based for

Ключевое слово Переменная Контейнер Тело
`for(auto i : array) std::cout << i;`

Ключевое слово Переменный Выражение2 Тело

↓ ↓ ↓ ↓

for(auto [key, value] : mymap) std::cout << i;

for (range-declaration : range-expression)

range-expression – любое выражение, представляющее последовательность элементов (либо массив, либо объект, для которого определены методы или функции `begin` и `end`) или список инициализации.

range-declaration – объявление именованной переменной, тип которой является типом элемента последовательности, представленного **range-expression**, или ссылкой на этот тип. Часто использует спецификатор `auto` для автоматического определения типа.

for(инициализация; range-declaration : range-expression)

```
for(auto list = {1,2,3}; auto i : list){  
    std::cout << i;  
}
```

for и области видимости

Также, как и `if` и `while`, так и `for` создаёт свою область видимости.

```
int main() {  
    for (int i=0; i<10; i++){  
        int i = -1; // Не OK, нарушаем ODR  
        cout << i;  
    }  
}
```

```
int main() {  
    for (int i=0; i<10; i++){  
        {  
            int i = -1; // OK  
            cout << i;  
        }  
    }  
}
```

Jump Statements (Стейтменты перехода)

В стандарте указано 5 видов:

- `break`;
- `continue`;
- `return`. Прерывание и выход из функции (будет рассмотрен позже);
- `co_return`. Приостановка и выход из корутины (будет рассмотрен позже);
- `goto`.

break

Разрешено использовать только в switch и циклах. Приводит к немедленному выходу за пределы switch или цикла соответственно.

```
for (int i=0; i<10; i++){  
    cout << 1;  
    break;  
    cout << 2;  
}
```



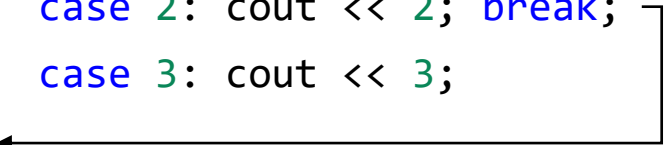
```
while(true){  
    cout << 1;  
    break;  
    cout << 2;  
}
```



```
do{  
    cout << 1;  
    break;  
    cout << 2;  
}while(true);
```



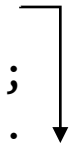
```
switch(i){  
    case 1: cout << 1;  
    case 2: cout << 2; break;  
    case 3: cout << 3;  
}
```



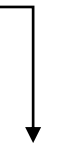
continue

Разрешено использовать только в циклах. Приводит к пропуску кода, расположенного после `continue` и до конца тела цикла. Цикл не прерывается.

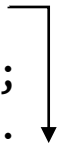
```
for (int i=0; i<10; i++){  
    cout << 1;  
    continue;  
    cout << 2;  
    cout << 3;  
}
```



```
while(true){  
    cout << 1;  
    continue;  
    cout << 2;  
    cout << 3;  
}
```



```
do{  
    cout << 1;  
    continue;  
    cout << 2;  
    cout << 3;  
}while(true);
```



Jump Statement goto (Стейтмент перехода goto)

```
label: // метка  
      /* код */  
goto label;
```

* label – обычный идентификатор

Свойства метки goto

- Метка – это идентификатор, подчиняющийся тем же правилам именования, что и переменные.
- Метки имеют собственное пространство имён – они не конфликтуют с именами переменных, функций и типов.

```
int main() {  
    int start = 5;  
    start:           // OK: метка и переменная – разные сущности  
    goto start;  
}
```

- Метки не могут быть вложены в другие метки с тем же именем в одной функции – повторение имени метки в одной функции ошибка.

Свойства метки goto

- Метка видима во всей функции, а не только в блоке, где она объявлена. Это единственное исключение из правила «область видимости = блок».

```
int main() {  
    goto end;           // OK: метка end видна отсюда  
    {  
        end:           // метка внутри вложенного блока  
        ;  
    }  
}
```

- Метка не обязана использоваться. Неиспользуемая метка – не ошибка.

Свойства метки goto - ограничения

- Нельзя прыгать из функции в другую функцию;
- Прыгать в обработчик исключения (catch) или из него;
- Прыгать внутрь блока в обход объявлений с нетривиальной инициализацией:

```
int main(){
    goto skip;
    int x = 5;    // ОШИБКА: скалярный тип, но есть инициализатор
    // int x;     // ОК: скалярный тип, без инициализации
    // string s;  // ОШИБКА: класс с нетривиальным конструктором
skip:
    ;
}
```

Базовый ввод вывод

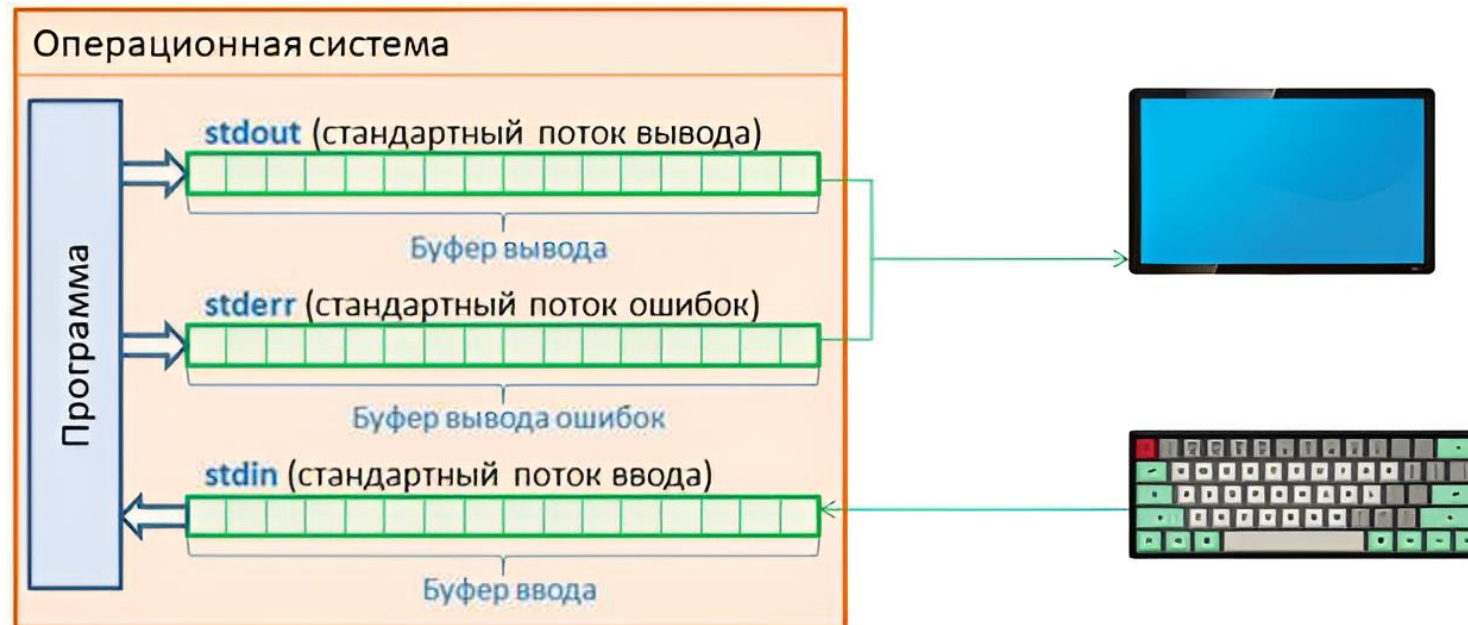
Потоки и стандартные устройства

Стандартные потоки:

- `std::cin / stdin` — стандартный ввод;
- `std::cout / stdout` — стандартный вывод;
- `std::cerr / stderr` — поток ошибок;
- `std::clog` — буферизованный поток ошибок в C++.

Буферизация

Буферизация – данные накапливаются в промежуточном хранилище и выводятся/вводятся позже, при заполнении буфера или принудительно, командой `flush`. Поток ошибок, по умолчанию не буферизируется, т.е. после каждой команды вывода, выполняется автоматический `flush`.



std::cout – ВЫВОД

- `iostream` – для ввода-вывода;
- `<<` – оператор вставки в поток;
- Возвращает поток, поэтому можно строить цепочки;

```
#include <iostream>

int main() {
    std::cout << "Hello, World!";
}
```

```
#include <iostream>

int main() {
    int age = 20;
    std::cout << "Age: " << age << '\n';
}
```

std::endl или '\n'

- в "" – строки;
- В ' ' – одиночные символы и эскейп последовательности;
- '\n' – перевод вывода на новую строку;
- std::flush – принудительный сброс буфера.

```
std::cout << "Hello" << std::endl; // '\n' + flush
```

```
std::cout << "Hello" << '\n';      // только '\n'
```

```
std::cout << "No newline" << std::flush;
```

Вывод данных на экран – долгая операция (т.к. нужно «дёрнуть» ОС). Чаще дёргаешь – больше времени тратишь. Вывод: реже используйте std::endl чаще обычный '\n'.

Буфер ускоряет работу программы, т.к. уменьшает количество обращений к ОС, но при неожиданном завершении работы программы, данные из буфера не попадут на экран.

std::cin — ВВОД

- `istream` — для ввода-вывода;
- `>>` — оператор извлечения из потока;
- Возвращает поток ввода, поэтому можно строить цепочки;
- Пропускает ведущие пробельные символы (пробелы, табуляции, переносы строк) удаляя их из потока ввода;
- Разделение данных происходит по пробельным символам;
- Автоматически парсит данные в зависимости от типа переменной;
- Считанные данные удаляются из потока, не считанные остаются в потоке и ждут следующего чтения.

```
int x;  
double y;  
string str;  
std::cin >> x >> y >> str;
```

>> или std::getline

- `string` – для `std::string` и `std::getline`;
- `>>` – игнорирует пробельные символы перед и читает до ближайшего пробельного символа;
- `std::getline` – читает до ближайшего символа разделителя, по умолчанию `'\n'`;
- `std::getline` – удаляет символ разделитель из потока;
- `>>` – удаляет пробельные символы в начале, но в конце (если есть) оставляет в потоке. Может быть проблемой при смешивании разных вариантов ввода.

```
std::string word, line, part;
```

```
std::cin >> word; // до пробельного символа
```

```
std::getline(std::cin, line); // до '\n'
```

```
std::getline(std::cin, part, ','); // до ','
```

std::cin.ignore

// Удаляем из потока один символ

```
std::cin.ignore();
```

// Удаляем из потока всё пока не достигнем '\n' (тоже удаляется) или конца

```
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

// Удаляем из потока до 10 символов, пока не достигнем '.' (тоже удаляется) или конца

```
std::cin.ignore(10, '.');
```

Манипуляторы <iomanip>

```
#include <iostream>
#include <iomanip>

int main() {
    // Пример 1: setw и setfill – задаём ширину поля и символ заполнения
    // setw действует только на следующий выводимый элемент, setfill сохраняется до смены
    std::cout << std::setfill('*') << std::setw(10) << 7 << "\n"; // *****7
    std::cout << std::setw(10) << "Hi" << "\n"; // *****Hi (fill остаётся '*')

    // Пример 2: выравнивание left/right внутри поля заданной ширины
    std::cout << std::setw(12) << std::right << 123 << "\n"; // |      123|
    std::cout << std::setw(12) << std::left << 123 << "\n"; // |123      |

    // Пример 3: fixed, scientific, setprecision – формат и точность чисел
    double x = 1234.5678;
    std::cout << std::fixed << std::setprecision(2) << x << "\n"; // 1234.57 (фиксированный, 2 знака после запятой)
    std::cout << std::scientific << std::setprecision(2) << x << "\n"; // 1.23e+03 (экспоненциальный, 2 знака в мантиссе)

    // Пример 4: hex, dec, oct – системы счисления для целых чисел
    int n = 255;
    std::cout << std::dec << n << "\n"; // 255
    std::cout << std::hex << n << "\n"; // ff
    std::cout << std::oct << n << "\n"; // 377

    // Пример 5: boolalpha – вывод булевых значений как true/false
    bool b = true;
    std::cout << b << "\n"; // 1
    std::cout << std::boolalpha << b << "\n"; // true
    std::cout << std::noboolalpha << b << "\n"; // снова 1

    // Пример 6: showpos и showpoint – отображение знака и десятичной точки
    int p = 5;
    std::cout << p << "\n"; // 5
    std::cout << std::showpos << p << "\n"; // +5

    double z = 10.0;
    std::cout << z << "\n"; // 10
    std::cout << std::fixed << std::setprecision(3) << z << "\n"; // 10.000 (всегда с точкой)
    std::cout << std::showpoint << z << "\n"; // 10.000 (всегда с точкой)
}
```